

# Verifying ABA with Leslie

Gaspard Reghem

June 5, 2026

### **Abstract**

Our main objective is the verification of an implementation of Asynchronous Byzantine Agreement. The proof of correctness consist in building a simulation relation between the implementation and an implementation running in an idealised setting. The simulation ensures that the correctness of the idealised implementation implies the correctness of the original one. However, this implementation is complex, thus, building the simulation in one go is intractable. Fortunately, the implementation is designed as the interlocking of multiple sub-protocols (GBCA, WCC, RSD, Gather, AVSS, BRB). This architecture allows us to decompose the construction of the overall simulation into smaller steps. Each implementation of sub-protocols will be simulated by an idealised version. Thus, when building the simulation for a protocol using sub-protocols, the sub-implementations will be replaced by their idealised versions; making the construction easier.

# 1 Preliminaries

This section gathers the standard material used in the rest of the document: probabilistic labelled transition systems, Segala-style relation lifting, weak transitions and probabilistic forward simulation, the specialisation to the non-probabilistic setting, partial-observation adversaries together with the belief construction, and CSP-style parallel composition together with its compositionality results. Throughout,  $\mathcal{D}(X)$  denotes the set of (discrete) probability distributions over a set  $X$ , and  $\bar{\mu} \subseteq X$  denotes the support of  $\mu \in \mathcal{D}(X)$ .

## 1.1 Probabilistic Labelled Transition Systems

**Definition 1** (Probabilistic Labelled Transition System). A *probabilistic labelled transition system* (PLTS) is a tuple  $\mathcal{P} = (Q, L, T, q_0)$  where

- $Q$  is a (countable) set of *states*;
- $L$  is a set of *labels*;
- $T \subseteq Q \times L \times \mathcal{D}(Q)$  is a set of *probabilistic transitions*;
- $q_0 \in Q$  is the *initial state*.

A transition  $(q, l, \mu) \in T$  is written  $q \xrightarrow{l} \mu$  and is said to be *enabled in  $q$* . The label set is partitioned as  $L = L_{\text{int}} \sqcup L_{\text{ext}}$  into *internal* and *external* labels.

**Definition 2** (Execution and trace). An *execution* of  $\mathcal{P} = (Q, L, T, q_0)$  is a finite or infinite alternating sequence

$$e \in (Q \cdot L)^* \cdot Q \cup (Q \cdot L)^\omega$$

such that  $e_0 = q_0$  and, whenever  $2i + 2 < |e|$ , there exists  $\mu \in \mathcal{D}(Q)$  with  $(e_{2i}, e_{2i+1}, \mu) \in T$  and  $e_{2i+2} \in \bar{\mu}$ . The *trace* of an execution  $e$ , written  $tr(e)$ , is the projection of  $e$  onto  $L_{\text{ext}}$ .

**Definition 3** (Scheduler). A *scheduler* for  $\mathcal{P}$  is a map  $\sigma$  sending every finite execution  $e$  to a sub-distribution over transitions whose support consists of transitions enabled in the last state of  $e$ .

**Definition 4** (Probabilistic execution). Given a scheduler  $\sigma$  and a starting state  $q$ , the *probabilistic execution*  $\mathbb{P}_{\sigma, q}$  is the unique measure on the  $\sigma$ -algebra generated by cones of executions characterised by

- $\mathbb{P}_{\sigma, q}(C_q) = 1$  and  $\mathbb{P}_{\sigma, q}(C_{q'}) = 0$  for any  $q' \neq q$ ;
- for  $e' \cdot l \cdot q'$  a finite execution,

$$\mathbb{P}_{\sigma, q}(C_{e' \cdot l \cdot q'}) = \mathbb{P}_{\sigma, q}(C_{e'}) \cdot \sigma(e')(l) \cdot \mu(q'),$$

where  $t = (last(e'), l, \mu)$  is the (necessarily unique) transition picked by  $\sigma$  with label  $l$ .

**Definition 5** (Trace distribution). The trace function  $tr$  is measurable from the  $\sigma$ -algebra of cones of executions to the  $\sigma$ -algebra of cones of traces. The *trace distribution* induced by  $\sigma$  from  $q$  is the pushforward  $tr_* \mathbb{P}_{\sigma, q}$ .

**Definition 6** (Trace distribution pre-order). Let  $\mathcal{P}_1$  and  $\mathcal{P}_2$  be two PLTS sharing the same set of external labels. We write  $\mathcal{P}_1 \sqsubseteq_{\text{TD}} \mathcal{P}_2$  when every trace distribution achievable by  $\mathcal{P}_1$  from its initial state is also achievable by  $\mathcal{P}_2$  from its initial state. This defines the *trace distribution pre-order*.

## 1.2 Probabilistic Forward Simulation

The trace distribution pre-order is a semantic object that is hard to manipulate directly. The standard tool to establish it is Segala's *probabilistic forward simulation*, which lifts a state-to-distribution relation through the transition structure. We first need to lift relations between sets to relations between distributions.

**Definition 7** (Lifting of a relation between sets). Let  $R \subseteq A \times B$ . The *lifting* of  $R$  is the relation  $R^\# \subseteq \mathcal{D}(A) \times \mathcal{D}(B)$  defined by  $(\mu_A, \mu_B) \in R^\#$  iff there exists a weight function  $w : A \times B \rightarrow [0, 1]$  such that

1.  $w(a, b) > 0 \implies (a, b) \in R$ ;
2.  $\mu_A(a) = \sum_{b \in B} w(a, b)$  for every  $a \in A$ ;
3.  $\mu_B(b) = \sum_{a \in A} w(a, b)$  for every  $b \in B$ .

**Definition 8** (Lifting of a measure-valued relation). Let  $R \subseteq A \times \mathcal{D}(B)$ . The *lifting*  $R^\# \subseteq \mathcal{D}(A) \times \mathcal{D}(B)$  is defined by  $(\mu_A, \mu_B) \in R^\#$  iff there exists a family  $(\mu_B^a)_{a \in \bar{\mu}_A}$  such that

1.  $(a, \mu_B^a) \in R$  for every  $a \in \bar{\mu}_A$ ;
2.  $\mu_B = \sum_{a \in A} \mu_A(a) \cdot \mu_B^a$ .

**Definition 9** (Hyper-transition). For every label  $l \in L$ , the relation  $\{(q, \mu) \mid (q, l, \mu) \in T\} \subseteq Q \times \mathcal{D}(Q)$  lifts (via Definition 8) to a relation  $\xrightarrow{l} \subseteq \mathcal{D}(Q) \times \mathcal{D}(Q)$ , called the *hyper-transition* relation.

**Definition 10** (Weak transitions). A scheduler that schedules only transitions with internal labels and stops almost-surely defines a *weak internal transition*  $q \Rightarrow \mu$ , where  $\mu$  is the distribution on stopping states induced by the scheduler started in  $q$ . By Definition 8 the relation  $\Rightarrow$  extends to  $\mathcal{D}(Q) \times \mathcal{D}(Q)$ . For an external label  $l \in L_{\text{ext}}$ , the *weak external transition*  $q \xRightarrow{l} \mu$  holds whenever there exist  $\mu_1, \mu_2 \in \mathcal{D}(Q)$  with

$$q \Rightarrow \mu_1 \xrightarrow{l} \mu_2 \Rightarrow \mu.$$

**Definition 11** (Probabilistic forward simulation). A *probabilistic forward simulation* of  $\mathcal{P}_1$  by  $\mathcal{P}_2$  is a relation  $S \subseteq Q_1 \times \mathcal{D}(Q_2)$  such that  $(q_0^1, \delta_{q_0^2}) \in S$  and, for every  $(q_1, \mu_2) \in S$  and every transition  $q_1 \xrightarrow{l} \mu_1$  of  $\mathcal{P}_1$ :

1. if  $l$  is internal, there exists  $\mu'_2$  with  $\mu_2 \Rightarrow \mu'_2$  and  $(\mu_1, \mu'_2) \in S^\#$ ;
2. if  $l$  is external, there exist  $\mu'_2$  with  $\mu_2 \xRightarrow{l} \mu'_2$  and  $(\mu_1, \mu'_2) \in S^\#$ .

We write  $\mathcal{P}_1 \preceq_{\text{PFS}} \mathcal{P}_2$  when such a simulation exists.

**Theorem 12** (Soundness of probabilistic forward simulation). *If  $\mathcal{P}_1 \preceq_{\text{PFS}} \mathcal{P}_2$  then  $\mathcal{P}_1 \sqsubseteq_{\text{TD}} \mathcal{P}_2$ .*

**Lemma 13** (Transitivity of probabilistic forward simulation). *Probabilistic forward simulation is transitive: if  $\mathcal{P}_1 \preceq_{\text{PFS}} \mathcal{P}_2$  and  $\mathcal{P}_2 \preceq_{\text{PFS}} \mathcal{P}_3$  then  $\mathcal{P}_1 \preceq_{\text{PFS}} \mathcal{P}_3$ . Concretely, given witnesses  $S_{12} \subseteq Q_1 \times \mathcal{D}(Q_2)$  and  $S_{23} \subseteq Q_2 \times \mathcal{D}(Q_3)$ , the composed relation*

$$S_{13} := \{(q_1, \mu_3) \mid \exists \mu_2, (q_1, \mu_2) \in S_{12} \text{ and } (\mu_2, \mu_3) \in S_{23}^\#\}$$

*is a probabilistic forward simulation of  $\mathcal{P}_1$  by  $\mathcal{P}_3$ .*

Note that  $\preceq_{\text{PFS}}$  is also reflexive (the diagonal  $\{(q, \delta_q)\}$  is a simulation), hence a pre-order; Lemma 13 is the part actually used in the proof decomposition.

### 1.3 Equivalence in the Non-Probabilistic Setting

When all transitions lead to a Dirac distribution, the PLTS framework collapses to the classical (non-probabilistic) labelled transition system, and the lifted notions admit a more elementary description.

**Definition 14** (Labelled transition system). A PLTS is a *labelled transition system* (LTS) if every transition's target is a Dirac distribution. Equivalently, its transition relation can be viewed as a subset  $T \subseteq Q \times L \times Q$ .

**Definition 15** (Weak transitions on an LTS). On an LTS, the weak internal transition  $\Rightarrow$  coincides with the reflexive transitive closure of internal transitions. The weak external transition is  $q \xRightarrow{l} q'$  iff there exist  $q_1, q_2$  with  $q \Rightarrow q_1 \xrightarrow{l} q_2 \Rightarrow q'$ .

**Definition 16** (Forward simulation). A *forward simulation* of an LTS  $\mathcal{L}_1$  by an LTS  $\mathcal{L}_2$  is a relation  $R \subseteq Q_1 \times Q_2$  with  $(q_0^1, q_0^2) \in R$  such that, for every  $(q_1, q_2) \in R$  and every  $q_1 \xrightarrow{l} q_1'$ :

1. if  $l$  is internal, there exists  $q_2'$  with  $q_2 \Rightarrow q_2'$  and  $(q_1', q_2') \in R$ ;
2. if  $l$  is external, there exist  $q_2'$  with  $q_2 \xRightarrow{l} q_2'$  and  $(q_1', q_2') \in R$ .

We write  $\mathcal{L}_1 \preceq_{\text{FS}} \mathcal{L}_2$  when such a simulation exists.

**Proposition 17** (Equivalence on LTS). *On LTS, forward simulation and probabilistic forward simulation coincide:  $\mathcal{L}_1 \preceq_{\text{FS}} \mathcal{L}_2$  iff  $\mathcal{L}_1 \preceq_{\text{PFS}} \mathcal{L}_2$ .*

### 1.4 Partial-Observation Adversaries

The scheduler of Definition 3 models an *omniscient* adversary that can inspect the entire execution. To model weaker adversaries one restricts schedulers to depend only on a partial view of the system, defined via *signals*.

**Definition 18** (Partial observation). A *partial observation* on a PLTS  $\mathcal{P}$  is a pair of mappings

$$obs_Q: Q \longrightarrow \Sigma_Q, \quad obs_L: T \longrightarrow \Sigma_L,$$

sending states and transitions to their *state signal* and *label signal* in two abstract signal sets  $\Sigma_Q, \Sigma_L$ .

**Definition 19** (Consistent scheduler). A scheduler  $\sigma$  is *consistent* with the partial observation  $(obs_Q, obs_L)$  if its output depends on its input only through signals: two finite executions with the same image under the observation produce schedulers that coincide on the corresponding label signals.

To prevent pathological signal choices that would conflate genuinely distinct non-deterministic alternatives, we restrict to admissible observations.

**Definition 20** (Admissible observation). A partial observation is *admissible* when

1. the state signal  $obs_Q(q)$  determines the set  $\{obs_L(t) \mid t \text{ enabled in } q\}$  of label signals of enabled transitions;
2. in any state  $q$ , no two distinct enabled transitions share the same label signal.

Under these conditions the adversary always knows which non-deterministic choices are available; partial information arises solely from the probabilistic outcomes of transitions.

**Definition 21** (Belief PLTS). Let  $\mathcal{P} = (Q, L, T, q_0)$  be a PLTS equipped with an admissible partial observation. Its *belief PLTS*  $\mathcal{B}(\mathcal{P})$  is the PLTS whose

- states are distributions over sets of states sharing a common state signal;
- labels are the label signals  $\Sigma_L$ ;
- transitions describe the update of the current belief when an underlying transition fires;
- initial state is the Dirac on  $q_0$ .

**Theorem 22** (Belief reduction). *The trace distributions reachable on  $\mathcal{P}$  by schedulers consistent with the partial observation coincide with the trace distributions reachable on  $\mathcal{B}(\mathcal{P})$  by arbitrary (omniscient) schedulers.*

Theorem 22 is the practical lever behind our use of partial information: the soundness of probabilistic forward simulation (Theorem 12) is stated for omniscient schedulers, and the belief construction lets us reuse it in the partial-information setting.

## 1.5 Parallel Composition

**Definition 23** (CSP-style parallel composition). The *parallel composition*  $\mathcal{P}_1 \parallel \mathcal{P}_2$  is the CSP-style synchronous product of  $\mathcal{P}_1$  and  $\mathcal{P}_2$ : the two components synchronise on the external labels they share and interleave on all other transitions.

**Theorem 24** (Pre-congruence). *Probabilistic forward simulation is a pre-congruence for parallel composition: if  $\mathcal{P}_1 \preceq_{\text{PFS}} \mathcal{P}'_1$  then  $\mathcal{P}_1 \parallel \mathcal{P}_2 \preceq_{\text{PFS}} \mathcal{P}'_1 \parallel \mathcal{P}_2$  for every  $\mathcal{P}_2$ . Moreover, partial observations on the components extend canonically to the composition.*

**Proposition 25** (Failure of compositionality of  $\sqsubseteq_{\text{TD}}$ ). *The trace distribution pre-order is not a pre-congruence for parallel composition: there exist PLTS  $\mathcal{P}_1, \mathcal{P}'_1, \mathcal{P}_2$  with  $\mathcal{P}_1 \sqsubseteq_{\text{TD}} \mathcal{P}'_1$  such that  $\mathcal{P}_1 \parallel \mathcal{P}_2 \not\sqsubseteq_{\text{TD}} \mathcal{P}'_1 \parallel \mathcal{P}_2$ .*

**Theorem 26** (Largest pre-congruence inside  $\sqsubseteq_{\text{TD}}$ ). *Probabilistic forward simulation characterises the largest pre-congruence (for CSP-style parallel composition) contained in the trace distribution pre-order. In particular, the simulation pre-order  $\preceq_{\text{PFS}}$  is contained in  $\sqsubseteq_{\text{TD}}$ , and every pre-congruence contained in  $\sqsubseteq_{\text{TD}}$  is contained in  $\preceq_{\text{PFS}}$ .*

## 2 List of the Protocols

**Common Notation.** Throughout this section,  $Id$  denotes the set of process identifiers with  $n := |Id|$ ,  $F \subseteq Id$  denotes the (a priori unknown) set of faulty processes with  $f \geq |F|$  a fixed upper bound, and  $Id \setminus F$  is the set of correct processes. For each protocol we provide two implementations:

- an *ideal implementation* (**Spec**), designed for a no-delay system with crash failures and mediated by a trusted third party;
- a *concrete implementation* (**Impl**), designed for an asynchronous system with Byzantine failures.

Messages exchanged with the third party (resp. between processes) are written  $\langle \text{TAG}, \cdot \rangle$ , state variables stay in plain math, and arrays indexed by process identifiers are written  $v[id]$ .

## 2.1 Main Protocol: Asynchronous Byzantine Agreement (ABA)

(taken from [ABDY22]) Processes start with a bit  $\{0, 1\}$  and may return a bit. A program implements ABA if it satisfies three properties: Validity, Agreement, Almost-Sure Termination.

- Validity: if a correct process returns  $b$  then a correct process had  $1 - b$  as input.
- Agreement: if two correct processes return, then their outputs are equal.
- Termination: if  $n - f$  correct processes take part in the protocol then, with probability 1, all correct processes will eventually return.

**Protocol 1.** *Ideal implementation.* The trusted third party collects each process's input bit and, once at least  $n - f$  contributions support some bit  $b$ , fixes the bound value to  $b$  and broadcasts  $\langle \text{OUTPUT}, b \rangle$  to all processes. Late callers are coerced to the bound value or to a random draw from a non-Dirac distribution  $\mu$ , which preserves the bound while modelling the adversary's ability to skew honest participation.

---

### Algorithm 1 Ideal Implementation of ABA

---

**Input:**  $b \in \{0, 1\}$

**Code for every process:**

send  $\langle \text{INPUT}, b \rangle$  to the third party

**upon** receiving  $\langle \text{OUTPUT}, b \rangle$  from the third party:

return  $b$

**Code for the third party:**

$bind, call \leftarrow \perp, (\perp)^{Id}$

**upon** receiving  $\langle \text{INPUT}, b \rangle$  from a process  $id$  and  $bind = \perp$ :

$call[id] \leftarrow b$

**upon**  $|F \cup \{id \in Id \setminus F \mid call[id] = b\}| \geq n - f$ :

$bind, call \leftarrow b, (\perp)^{Id}$

**if**  $\{id \in Id \setminus F \mid call[id] = 1 - b\} = \emptyset$ :

send  $\langle \text{OUTPUT}, b \rangle$  to all

**upon**  $call[id] = \perp$  and  $bind \neq \perp$ :

$call[id] \leftarrow bind$  **or**  $\mu$

$\triangleright \mu$  is any non-Dirac distribution over  $\{0, 1\}$

---

**Protocol 2.** *Concrete implementation.* Round-based protocol combining Graded Binding Crusader Agreement (GBCA) with a Weak Common Coin (WCC). Each round, the process refines its bit through GBCA and tosses a coin via WCC; it adopts the coin if GBCA was undecided ( $b = \perp$ ) and otherwise decides on  $b$  when GBCA returned the highest grade  $A$ , while continuing to run to help the others terminate.

**Definition 27.**  $\text{ABA.Concrete} := \text{ABA.Impl}[\text{GBCA.Impl}, \text{WCC.Impl}]$ , the instantiation of  $\text{ABA.Impl}$  with concrete implementations of the sub-protocols.

**Definition 28.**  $\text{ABA.Hybrid} := \text{ABA.Impl}[\text{GBCA.Spec}, \text{WCC.Spec}]$ , the instantiation of  $\text{ABA.Impl}$  with ideal specifications of the sub-protocols.

## 2.2 ABA Sub-Protocols

**Graded Binding Crusader Agreement (GBCA)** (taken from [ABDY22, AW23]) Processes start with a binary input  $\{0, 1\}$  and may return a value in  $\{(0, A), (0, B), (\perp, C), (1, B), (1, A)\}$ . A

---

**Algorithm 2** Implementation of ABA taken from [ABDY22]

---

**Input:**  $b \in \{0, 1\}$

$r \leftarrow 1$

$decided \leftarrow \perp$

**loop**

$(b, g) \leftarrow \text{GBCA}_r(b)$   $\triangleright b \in \{0, 1, \perp\}, g \in \{A, B, C\}$

$c \leftarrow \text{WCC}_r()$   $\triangleright$  common coin for round  $r$

**if**  $b = \perp$  **then**

$b \leftarrow c$

**else if**  $g = A$  **then**

$decided \leftarrow b$   $\triangleright$  keep participating to help others

**end if**

$r \leftarrow r + 1$

**end loop**

$\triangleright$  TODO: verify termination rule

---

program implements GBCA if it satisfies four properties: Validity, Graded Agreement, Binding, Termination.

- **Validity:** if a correct process does not return  $(b, A)$  then a correct process had  $1 - b$  as input.
- **Graded Agreement:** if two correct processes return  $(b, X)$  and  $(b', X')$  then  $b = 0 \implies b' \neq 1$  and  $X = A \implies X' \neq \perp$ .
- **Binding:** when a correct process returns then a bound value  $b$  is defined and, in all extensions of the execution, correct processes do not return  $(1 - b, B)$  or  $(1 - b, A)$ .
- **Termination:** if  $n - f$  correct processes take part in the protocol then all correct processes will eventually return.

**Protocol 3. Ideal implementation.** The trusted third party tallies the contributions and, as soon as a quorum supports some bit  $b$ , fixes  $bind$  to  $b$  and sets the global grade:  $A$  if no correct process input  $1 - b$ , otherwise  $C$ . Each process is then served either the highest grade pair  $(bind, A)$  (resp.  $(\perp, C)$ ) or the conservative  $(bind, B)$  as chosen non-deterministically by the adversary.

**Protocol 4. Concrete implementation.** Requires no sub-protocol. Parameters:  $n$  processes with at most  $f < n/3$  corrupted, each holding an input bit  $b \in \{0, 1\}$ . After an input-amplification phase, the protocol runs three Bracha-style rounds of  $\langle \text{ECHO}, \cdot \rangle$ ,  $\langle \text{VOTE}, \cdot \rangle$ ,  $\langle \text{BIND}, \cdot \rangle$  messages to settle the bound value and assign the grade.

**Weak Common Coin (WCC)** Processes start with no input and may return a bit. A program implements WCC if it satisfies three properties:  $\epsilon$ -Goodness, Unpredictability, Termination.

- $\epsilon$ -Goodness: with probability  $\epsilon$ , all correct processes return 0 (resp. 1).
- Unpredictability: until  $f + 1$  processes have called WCC, all outcomes are still possible.
- Termination: if  $n - f$  correct processes take part in the protocol then all correct processes will eventually return.



---

**Algorithm 3** Ideal Implementation of GBCA

---

**Input:**  $b \in \{0, 1\}$

**Code for every process:**

send  $\langle \text{INPUT}, b \rangle$  to the third party

**upon** receiving  $\langle \text{OUTPUT}, (b, g) \rangle$  from the third party:  
return  $(b, g)$

**Code for the third party:**

$bind, grade, call \leftarrow \perp, \perp, (\perp)^{Id}$

**upon** receiving  $\langle \text{INPUT}, b \rangle$  from a process  $id$ :

$call[id] \leftarrow b$

**upon**  $|F \cup \{id \in Id \setminus F \mid call[id] = b\}| \geq n - f$ :

$bind \leftarrow b$

**if**  $\{id \in Id \setminus F \mid call[id] = 1 - b\} = \emptyset$ :

$grade \leftarrow A$

**else**

$grade \leftarrow C$

**upon**  $bind \neq \perp$  and  $grade = A$  and not sent  $\langle \text{OUTPUT}, \cdot \rangle$  to process  $id$ :

send  $\langle \text{OUTPUT}, (bind, B) \rangle$  or  $\langle \text{OUTPUT}, (bind, A) \rangle$  to process  $id$

**upon**  $bind \neq \perp$  and  $grade = C$  and not sent  $\langle \text{OUTPUT}, \cdot \rangle$  to process  $id$ :

send  $\langle \text{OUTPUT}, (bind, B) \rangle$  or  $\langle \text{OUTPUT}, (\perp, C) \rangle$  to process  $id$

---

---

**Algorithm 4** Implementation of GBCA taken from [ABDY22]

---

**Input:**  $b \in \{0, 1\}$

send  $\langle \text{INPUT}, b \rangle$  to all processes

**upon** receiving  $\langle \text{INPUT}, b \rangle$  from at least  $f + 1$  processes and not sent  $\langle \text{INPUT}, b \rangle$  yet:  
send  $\langle \text{INPUT}, b \rangle$  to all

**upon** receiving  $\langle \text{INPUT}, b \rangle$  from at least  $n - f$  processes:

$Approved \leftarrow Approved \cup \{b\}$

**if** not sent  $\langle \text{ECHO}, \cdot \rangle$  yet **then** send  $\langle \text{ECHO}, b \rangle$  to all

wait until

(a) receiving  $\langle \text{ECHO}, b \rangle$  from at least  $n - f$  processes **or**

(b) receiving  $\langle \text{ECHO}, \cdot \rangle$  from at least  $n - f$  processes and  $|Approved| > 1$

**if** (a) **then** send  $\langle \text{VOTE}, b \rangle$  to all **else** send  $\langle \text{VOTE}, \perp \rangle$  to all

wait until

(a) receiving  $\langle \text{VOTE}, b \rangle$  from at least  $n - f$  processes **or**

(b) receiving  $\langle \text{VOTE}, \cdot \rangle$  from at least  $n - f$  processes and  $|Approved| > 1$

**if** (a) **then** send  $\langle \text{BIND}, b \rangle$  to all **else** send  $\langle \text{BIND}, \perp \rangle$  to all

wait until

(a) receiving  $\langle \text{BIND}, b \rangle$  from at least  $n - f$  processes **or**

(b) receiving  $\langle \text{BIND}, \cdot \rangle$  from at least  $n - f$  processes and there exists  $b \in \{0, 1\}$  such that  $\langle \text{BIND}, b \rangle$  has been received once and  $\langle \text{VOTE}, b \rangle$  has been received from  $f + 1$  processes and  $|Approved| > 1$  **or**

(c) receiving  $\langle \text{BIND}, \perp \rangle$  from at least  $n - f$  processes and  $|Approved| > 1$

**if** (a) **then** return  $(b, A)$  **elif** (b) **then** return  $(b, B)$  **else** return  $(\perp, C)$

---

**Protocol 5.** *Ideal implementation.* The trusted third party samples a coin  $r$  from a distribution  $\mu$  such that  $\mu(0) = \mu(1) = \epsilon$  and  $\mu(\perp) = 1 - 2\epsilon$ . If  $r \in \{0, 1\}$  the same value is broadcast to every process; otherwise each process is delivered an adversarially-chosen bit, modelling the worst case where the coin does not coordinate the honest parties.

---

**Algorithm 5** Ideal Implementation of WCC

---

**Input:** none

**Code for every process:**

send  $\langle \text{INPUT} \rangle$  to the third party

**upon** receiving  $\langle \text{OUTPUT}, b \rangle$  from the third party:

return  $b$

**Code for the third party:**

$r \leftarrow \mu$

$\triangleright \mu: 1 \mapsto \epsilon, 0 \mapsto \epsilon, \perp \mapsto 1 - 2\epsilon$

**if**  $r \neq \perp$ :

send  $\langle \text{OUTPUT}, r \rangle$  to all

**else**

for all  $id \in Id$ , send  $\langle \text{OUTPUT}, 0 \rangle$  **or**  $\langle \text{OUTPUT}, 1 \rangle$  to process  $id$

---

**Protocol 6.** *Concrete implementation.* Requires Gather and RSD. Details to be determined.

---

**Algorithm 6** Implementation of WCC (yet to be determined)

---

$x \leftarrow 0$

**if**  $y > 0$  **then**

loop

**end if**

---

**Definition 29.**  $\text{WCC.Concrete} := \text{WCC.Impl}[\text{Gather.Concrete}, \text{RSD.Concrete}]$ , the instantiation of  $\text{WCC.Impl}$  with fully concrete sub-protocols.

**Definition 30.**  $\text{WCC.Hybrid} := \text{WCC.Impl}[\text{Gather.Spec}, \text{RSD.Spec}]$ , the instantiation of  $\text{WCC.Impl}$  with ideal specifications of the sub-protocols.

## 2.3 WCC Sub-Protocols

**Gather** (taken from [SA21, SA24, AFW25]) Processes start with an input  $x \in X$  and may return a partial function  $f : \text{Proc} \rightharpoonup X$ . A program implements Gather if it satisfies four properties: Validity, Agreement, Common Core, Binding, Termination.

- **Validity:** if a correct process returns  $f$  and  $f(q)$  is defined and  $q$  is correct then  $f(q)$  is  $q$ 's input.
- **Agreement:** if two correct processes return  $f$  and  $g$  then  $f(q)$  and  $g(q)$  are equal or one of them is undefined.
- **Binding Common Core:** once a correct process has returned, there exists  $S \subseteq \text{Proc}$  of size at least  $n - f$  such that all  $f$  returned by a correct process in the future is defined on  $S$ .
- **Termination:** if  $n - f$  correct processes take part in Gather then all correct processes will eventually return.

**Protocol 7. Ideal implementation.** The trusted third party records each process's input, and once a quorum of correct contributions has been received, non-deterministically commits to a binding common core  $bind \subseteq Id$ . Each process is then served a partial function whose domain  $S$  is non-deterministically extended above  $bind$ , returning the recorded inputs of correct members of  $S$  and arbitrary values for corrupted members.

---

**Algorithm 7** Ideal Implementation of Gather

---

**Input:**  $m$

**Code for every process:**

send  $\langle \text{INPUT}, m \rangle$  to the third party

**upon** receiving  $\langle \text{OUTPUT}, f \rangle$  from the third party:

return  $f$

**Code for the third party:**

$call, bind \leftarrow \emptyset, \perp$

**upon** receiving  $\langle \text{INPUT}, m \rangle$  from process  $id$ :

$call \leftarrow call \cup \{(id, m)\}$

**upon**  $|F \cup \{id \in Id \setminus F \mid (id, \cdot) \in call\}| \geq n - f$  and  $bind = \perp$ :

$bind \leftarrow S$  for some  $S \subseteq F \cup \{j \in Id \setminus F \mid (j, \cdot) \in call\}$   $\triangleright$  Non-deterministic choice

**upon**  $bind \neq \perp$  and not sent  $\langle \text{OUTPUT}, \cdot \rangle$  to process  $id$ :

select  $bind \subseteq S \subseteq F \cup \{j \in Id \setminus F \mid (j, \cdot) \in call\}$  and

send  $\langle \text{OUTPUT}, \{(j, m) \in call \mid j \in S \setminus F\} \cup \{(j, \cdot) \mid j \in S \cap F\} \rangle$  to process  $id$

---

**Protocol 8. Concrete implementation.** Requires  $|Id|$  instances of BRB (process  $id$  is leader in  $BRB_{id}$ ). Each process first broadcasts its input via its own BRB instance, then amplifies the set of approved pairs  $AP$  through three rounds of  $\langle \text{ECHO}, \cdot \rangle$ ,  $\langle \text{VOTE}, \cdot \rangle$ ,  $\langle \text{BIND}, \cdot \rangle$  messages before returning the union of the approved sets.

---

**Algorithm 8** Implementation of Gather from [AFW25]

---

**Input:**  $m$

**Sub-Protocols:**  $|Id|$  instances of BRB (process  $id$  is leader in  $BRB_{id}$ )

$BRB_{id}.call(m)$

**upon**  $BRB_i.return(m')$ :

$AP \leftarrow AP \cup \{(i, m')\}$

**upon**  $|AP| \geq n - f$  and not sent  $\langle \text{ECHO}, \cdot \rangle$ :

send  $\langle \text{ECHO}, AP \rangle$  to all

**upon** receiving approved  $\langle \text{ECHO}, AP_{id} \rangle$  from at least  $n - f$  processes and not sent  $\langle \text{VOTE}, \cdot \rangle$ :

send  $\langle \text{VOTE}, \bigcup_{id} AP_{id} \rangle$  to all

**upon** receiving approved  $\langle \text{VOTE}, AP_{id} \rangle$  from at least  $n - f$  processes and not sent  $\langle \text{BIND}, \cdot \rangle$ :

send  $\langle \text{BIND}, \bigcup_{id} AP_{id} \rangle$  to all

**upon** receiving approved  $\langle \text{BIND}, AP_{id} \rangle$  from at least  $n - f$  processes:

return  $\bigcup_{id} AP_{id}$

---

**Definition 31.**  $\text{Gather.Concrete} := \text{Gather.Impl}[\text{BRB.Impl}]$ , the instantiation of  $\text{Gather.Impl}$  with the concrete implementation of BRB.

**Definition 32.**  $\text{Gather.Hybrid} := \text{Gather.Impl}[\text{BRB.Spec}]$ , the instantiation of  $\text{Gather.Impl}$  with the ideal specification of BRB.

**Random Secret Draw (RSD)** The RSD protocol is composed of two operations: Assignment and Retrieval. Assignment assigns a random, secret value to each process and Retrieval reveals these values to the processes. Moreover, the API of RSD provides a notification, for each process, recording which processes have a value assigned.

- **Assignment Validity:** if all correct processes call Assignment, then all correct processes will eventually receive a notification for every correct process.
- **Assignment Totality:** if a correct process receives a notification for process  $id$  then all correct processes will get such a notification.
- **Randomness:** if no correct process has called Retrieval then the value returned by Retrieval for any process follows the uniform distribution.
- **Secrecy:** if no correct process has called Retrieval then the adversary has no information about the return value of Retrieval.
- **Retrieval Termination:** if all correct processes call Retrieval then the operation terminates for all correct processes.

**Protocol 9. Ideal implementation.** The trusted third party tracks which processes have called Assignment; once a quorum is reached, it issues an  $\langle \text{ASSIGN}, id \rangle$  notification for every  $id \in Id$ . On Retrieval, after enough processes have asked, it samples a fresh value  $\mu$  uniformly over the output domain and delivers it to every process.

---

**Algorithm 9** Ideal Implementation of RSD

---

**Operation:**  $RSD.Asg()$

**Code for every process:**

send  $\langle \text{INPUT} \rangle$  to the third party.

**upon** receiving  $\langle \text{ASSIGN}, id \rangle$  from the third party:

**event**  $ValueAssigned(id)$

**Code for the third party:**

$call \leftarrow (\perp)^{Id}$

**upon** receiving  $\langle \text{INPUT} \rangle$  from process  $id$ :

$call[id] \leftarrow \top$

**upon**  $|F \cup \{id \in Id \setminus F \mid call[id] = \top\}| \geq n - f$ :

$\forall id \in Id$ , send  $\langle \text{ASSIGN}, id \rangle$  to all

**Operation:**  $RSD.Ret()$

**Code for every process:**

send  $\langle \text{RETRIEVE} \rangle$  to the third party.

**upon** receiving  $\langle \text{OUTPUT}, x \rangle$  from the third party:

    return  $x$

**Code for the third party:**

$call', out \leftarrow (\perp)^{Id}, (\perp)^{Id}$

**upon** receiving  $\langle \text{RETRIEVE} \rangle$  from process  $id$ :

$call'[id] \leftarrow \top$

**upon**  $|F \cup \{id \in Id \setminus F \mid call'[id] = \top\}| \geq n - f$  and  $out[id] = \perp$ :

$out[id] \leftarrow \mu$

    send  $\langle \text{OUTPUT}, out[id] \rangle$  to all

**Protocol 10. Concrete implementation.** Requires  $|Id|$  instances of BRB and  $|Id|^2$  instances of AVSS. Each process  $id$  draws a fresh secret  $x_{id'}$  for every  $id' \in Id$  and shares it through an AVSS instance, then uses its own BRB to publish the set  $S$  of dealers it received shares from. A process is assigned a value once it has collected, for each entry of the committed set  $S'$ , the corresponding AVSS sharing return; the retrieved value is the sum of the AVSS reconstructions.

---

**Algorithm 10** Implementation of RSD from [FKT22]

---

**Sub-Protocols:**  $|Id|$  instances of BRB (process  $id$  is leader in  $BRB_{id}$ ) and  $|Id|^2$  instances of AVSS (process  $id$  is dealer in  $AVSS_{id}^k$  for each  $k \in Id$ )

**Operation:**  $RSD.Asg()$

$S \leftarrow \emptyset$

for all  $id' \in Id$ :

$x_{id'} \leftarrow \mu$   $\triangleright \mu$ : uniform distribution over the domain of secrets  
 $AVSS_{id'}^{id'}.Sh.call(x_{id'})$

**upon**  $AVSS_{id'}^{id'}.Sh.return()$ :

$S \leftarrow S \cup \{id'\}$

**upon**  $|S| \geq n - f$  and not called  $BRB_{id}$ :

$BRB_{id}.call(S)$

**upon**  $BRB_{id'}.return(S')$  and  $\forall k \in S', AVSS_k^{id'}.Sh.return()$ :

$src[id'] \leftarrow S'$

**event**  $ValueAssigned(id')$

$\forall k \in S', AVSS_k^{id'}.Rec.call()$

**Operation:**  $RSD.Rec()$

return  $id' \mapsto \sum_{k \in src[id']} AVSS_k^{id'}.Rec.return()$

---

**Definition 33.**  $RSD.Concrete := RSD.Impl[BRB.Impl, AVSS.Spec]$ , the instantiation of  $RSD.Impl$  with the concrete implementation of BRB. AVSS is used as a black-box, so its specification is plugged in directly.

**Definition 34.**  $RSD.Hybrid := RSD.Impl[BRB.Spec, AVSS.Spec]$ , the instantiation of  $RSD.Impl$  with the ideal specifications of BRB and AVSS.

## 2.4 Basic Primitives

**Byzantine Reliable Broadcast (BRB)** (taken from [Bra87]) Each instance of BRB has a designated *leader* with an input message  $m$  and processes may return a received message. A program implements BRB if it satisfies three properties: Validity, Totality and Agreement.

- Validity: if the leader is not corrupted then all correct processes must return its input.
- Agreement: if two correct processes return then they return the same value.
- Totality: if a correct process returns then all correct processes eventually return.

**Protocol 11. Ideal implementation.** The trusted third party receives the leader's input and broadcasts it to every process when the leader is correct; otherwise it broadcasts an arbitrary message  $m'$  chosen non-deterministically.

---

**Algorithm 11** Ideal Implementation of BRB

---

**Input:**  $m \in \{0, 1\}^*$  for the leader

**Code for the leader:**

send  $\langle \text{INPUT}, m \rangle$  to the third party

**Code for every process:**

**upon** receiving  $\langle \text{OUTPUT}, m \rangle$  from the third party:  
return  $m$

**Code for the third party:**

**upon** receiving  $\langle \text{INPUT}, m \rangle$  from leader:

if leader is correct:

send  $\langle \text{OUTPUT}, m \rangle$  to all

else

send  $\langle \text{OUTPUT}, m' \rangle$  to all

$\triangleright$  Arbitrary  $m'$

---

**Protocol 12.** *Concrete implementation.* Requires no sub-protocol. Parameters:  $n$  processes with at most  $f < n/3$  corrupted; leader  $\ell$  with input  $m$ . Standard Bracha broadcast:  $\langle \text{INIT}, \cdot \rangle$  from the leader is echoed by all processes; an  $n - f$  echo quorum or an  $f + 1$  ready quorum triggers  $\langle \text{READY}, \cdot \rangle$ ; a  $2f + 1$  ready quorum triggers delivery.

---

**Algorithm 12** Implementation of BRB taken from [Bra87]

---

**Code for the leader  $\ell$  with input  $m$ :**

send  $\langle \text{INIT}, m \rangle$  to all processes

**Code for every process:**

**upon** first reception of  $\langle \text{INIT}, m \rangle$  from  $\ell$  **do**

send  $\langle \text{ECHO}, m \rangle$  to all processes

**upon** reception of  $\langle \text{ECHO}, m \rangle$  from at least  $n - f$  distinct processes **do**

if  $\langle \text{READY}, \cdot \rangle$  has not yet been sent **then**

send  $\langle \text{READY}, m \rangle$  to all processes

**upon** reception of  $\langle \text{READY}, m \rangle$  from at least  $f + 1$  distinct processes **do**

if  $\langle \text{READY}, \cdot \rangle$  has not yet been sent **then**

send  $\langle \text{READY}, m \rangle$  to all processes

**upon** reception of  $\langle \text{READY}, m \rangle$  from at least  $2f + 1$  distinct processes **do**

return  $m$

---

**Asynchronous Verifiable Secret Sharing (AVSS)** AVSS is actually a pair of protocols  $(Sh, Ret)$  with a designated *dealer* owning a secret  $s$ . A program implements a  $(1 - \epsilon)$ -correct AVSS if it satisfies six properties: Validity, Sharing Termination, Sharing Totality, Retrieval Termination, Commitment, Secrecy.

- Validity: if the dealer is correct and a correct process returns from *Ret* then the return value is the dealer's secret.
- Sharing Termination: if the dealer is correct and calls *Sh* then all correct processes will eventually return from *Sh*.

- **Sharing Totality:** if a correct process returns from  $Sh$  then all correct processes will eventually return from  $Sh$ .
- **Retrieval Termination:** if all correct processes call  $Ret$  then all correct processes will eventually return from  $Ret$ .
- **Commitment:** if a correct process returns from  $Sh$  then there exists a secret  $s$  such that correct processes can only retrieve  $s$  from that point.
- **Secrecy:** if the dealer is correct and no correct process has called  $Ret$  then the adversary has no information about the future retrieved value.

**Protocol 13.** *Ideal implementation.* The trusted third party stores the dealer's secret (or an adversarially-chosen one if the dealer is corrupted) and immediately notifies every process that sharing has completed. On Retrieval, as soon as more than  $f$  processes have asked, the stored secret is broadcast to all processes.

---

**Algorithm 13** Ideal Implementation of AVSS

---

**Input:**  $s \in \{0,1\}^*$  for the dealer

**Operation:**  $AVSS.Sh()$

**Code for the dealer:**

send  $\langle SH.INPUT, s \rangle$  to the third party

**Code for every process:**

**upon** receiving  $\langle SH.OUTPUT \rangle$  from the third party:  
return

**Code for the third party:**

**upon** receiving  $\langle SH.INPUT, s \rangle$  from dealer or dealer is corrupted:

$secret \leftarrow s$   $\triangleright s$  is chosen non-deterministically if no message received

send  $\langle SH.OUTPUT \rangle$  to all

**Operation:**  $AVSS.Ret()$

**Code for every process:**

send  $\langle RET.INPUT \rangle$  to the third party

**upon** receiving  $\langle RET.OUTPUT, m \rangle$  from the third party:

return  $m$

**Code for the third party:**

$call \leftarrow (\perp)^{Id}$

**upon** receiving  $\langle RET.INPUT \rangle$  from process  $id$ :

$call[id] \leftarrow \top$

**upon**  $|F \cup \{id \in Id \setminus F \mid call[id] = \top\}| \geq f + 1$ :

send  $\langle RET.OUTPUT, secret \rangle$  to all

---

AVSS is used as a black-box, so no concrete implementation is provided.

## 2.5 Summary of Properties

The linear safety properties are preserved by Segala's probabilistic forward simulation. The branching safety properties can be transformed into linear ones by enhancing the interface with

an external binding label. Segala’s simulation does not preserve liveness by default. However, it can be strengthened following [DSW22] to preserve liveness in a complete manner.

|        | Linear Safety        | Branching Safety    | Liveness                                                          |
|--------|----------------------|---------------------|-------------------------------------------------------------------|
| ABA    | Validity & Agreement | Binding             | Termination                                                       |
| GBCA   | Validity & Agreement |                     | Termination                                                       |
| WCC    | Validity & Agreement | Binding Common Core | Termination                                                       |
| Gather |                      |                     | Assignment Validity & Assignment Totality & Retrieval Termination |
| RSD    |                      |                     | Totality                                                          |
| BRB    | Validity & Agreement | Commitment          | Sharing Termination & Sharing Totality & Retrieval Termination    |
| AVSS   | Validity             |                     |                                                                   |

Table 1: Table of CTL Properties

Segala’s simulation was designed to be sound and complete for trace distributions so it preserves probabilistic safety properties. Note that Unpredictability is a probabilistic safety property. For probabilistic liveness, we will rely on the liveness preservation as it is sound (but not complete). The lack of completeness is not an issue because the limitation of the termination proof rule is probabilistic convergence, which is never dealt with by our simulations. The probabilistic convergence argument will only be necessary to prove that the ideal implementation of ABA almost-surely terminates and can be discharged to the super-Martingale proof rule of [EMMS26]. For Secrecy, we use the same trick as for branching: enhancing the interface.

|        | Probabilistic Safety                    | Probabilistic Liveness  | Secrecy |
|--------|-----------------------------------------|-------------------------|---------|
| ABA    | $\epsilon$ -Goodness & Unpredictability | Almost-Sure Termination |         |
| GBCA   |                                         |                         |         |
| WCC    |                                         |                         |         |
| Gather | Randomness                              |                         |         |
| RSD    |                                         |                         | Secrecy |
| BRB    |                                         |                         |         |
| AVSS   |                                         |                         | Secrecy |

Table 2: Table of non-CTL Properties

### 3 Proof Decomposition

The end goal is the correctness of `ABA.Impl`. We establish it via a simulation by `ABA.Spec` and transport the spec’s correctness. The simulation itself is decomposed recursively along the sub-protocol structure.

For a protocol  $P$  with concrete sub-protocols  $Q_1, \dots, Q_k$ , the step “ $P.Spec$  simulates  $P.Impl$ ” is split into:

1. (Substitution)  $P.Impl[Q_1.Spec, \dots, Q_k.Spec]$  is simulated by  $P.Impl$ , using the child simulations  $Q_i.Spec$  simulates  $Q_i.Impl$  and compositionality.



2. (Core)  $P.Spec$  simulates the hybrid  $P.Impl[Q_1.Spec, \dots, Q_k.Spec]$ .
3. (Transitivity) Compose (1) and (2).

Leaves of the recursion (no sub-protocol, or only the black-box  $AVSS.Spec$ ) skip the substitution step.

### 3.1 Top-level

**Theorem 35.** *ABA.Concrete satisfies Validity, Agreement and Almost-Sure Termination.*

*Proof.* Soundness of probabilistic forward simulation (Theorem 12) applied to 36 yields  $ABA.Concrete \sqsubseteq_{TD} ABA.Spec$ , which transports the safety properties (Validity and Agreement) of  $ABA.Spec$  to  $ABA.Concrete$ . Almost-Sure Termination is a probabilistic liveness property and is handled separately via the liveness-preserving strengthening of  $\preceq_{PFS}$ .  $\square$

### 3.2 ABA

**Theorem 36.** *ABA.Spec simulates ABA.Concrete.*

*Proof.* By transitivity of probabilistic forward simulation (Lemma 13), chain 38 with 37 by transitivity.  $\square$

**Lemma 37.** *ABA.Hybrid is simulated by ABA.Concrete.*

*Proof.*  $ABA.Concrete$  and  $ABA.Hybrid$  share the same skeleton  $ABA.Impl$  and differ only in their sub-protocol instances. Apply the pre-congruence of  $\preceq_{PFS}$  for parallel composition (Theorem 24) to the child simulations 42 and 39, lifted along the  $ABA.Impl$  context.  $\square$

**Lemma 38.** *ABA.Spec simulates ABA.Hybrid.*

### 3.3 WCC

**Theorem 39.** *WCC.Spec simulates WCC.Concrete.*

*Proof.* By transitivity of probabilistic forward simulation (Lemma 13), chain 41 with 40 by transitivity.  $\square$

**Lemma 40.** *WCC.Hybrid is simulated by WCC.Concrete.*

*Proof.*  $WCC.Concrete$  and  $WCC.Hybrid$  share the same skeleton  $WCC.Impl$  and differ only in the instances of Gather and RSD. Apply the pre-congruence of  $\preceq_{PFS}$  for parallel composition (Theorem 24) to the child simulations 43 and 46, lifted along the  $WCC.Impl$  context.  $\square$

**Lemma 41.** *WCC.Spec simulates WCC.Hybrid.*

### 3.4 GBCA

**Theorem 42.** *GBCA.Spec simulates GBCA.Impl. No sub-protocol: direct proof.*

### 3.5 Gather

**Theorem 43.** *Gather.Spec simulates Gather.Concrete.*

*Proof.* By transitivity of probabilistic forward simulation (Lemma 13), chain 45 with 44 by transitivity.  $\square$

**Lemma 44.** *Gather.Hybrid is simulated by Gather.Concrete.*

*Proof.* Gather.Concrete and Gather.Hybrid share the same skeleton Gather.Impl and differ only in the BRB instances. Apply the pre-congruence of  $\preceq_{\text{PFS}}$  for parallel composition (Theorem 24) to the child simulation 49, lifted along the Gather.Impl context (one instance per leader).  $\square$

**Lemma 45.** *Gather.Spec simulates Gather.Hybrid.*

### 3.6 RSD

**Theorem 46.** *RSD.Spec simulates RSD.Concrete.*

*Proof.* By transitivity of probabilistic forward simulation (Lemma 13), chain 48 with 47 by transitivity.  $\square$

**Lemma 47.** *RSD.Hybrid is simulated by RSD.Concrete. Only BRB is substituted; AVSS.Spec stays fixed on both sides.*

*Proof.* RSD.Concrete and RSD.Hybrid share the same skeleton RSD.Impl and the same AVSS.Spec instances; only the BRB instances are substituted. Apply the pre-congruence of  $\preceq_{\text{PFS}}$  for parallel composition (Theorem 24) to the child simulation 49, lifted along the RSD.Impl context (one BRB instance per leader).  $\square$

**Lemma 48.** *RSD.Spec simulates RSD.Hybrid.*

### 3.7 BRB

**Theorem 49.** *BRB.Spec simulates BRB.Impl. No sub-protocol: direct proof.*

# Bibliography

- [ABDY22] Ittai Abraham, Naama Ben-David, and Sravya Yandamuri. Efficient and adaptively secure asynchronous binary agreement via binding crusader agreement. In *41st ACM Symposium on Principles of Distributed Computing*, pages 381–391, 2022.
- [AFW25] Hagit Attiya, Itay Flam, and Jennifer L. Welch. Beyond canonical rounds: Communication abstractions for optimal byzantine resilience. *CoRR*, abs/2510.04310, 2025.
- [AW23] Hagit Attiya and Jennifer L. Welch. Multi-valued connected consensus: A new perspective on crusader agreement and adopt-commit. In *27th International Conference on Principles of Distributed Systems (OPODIS)*, 2023.
- [Bra87] Gabriel Bracha. Asynchronous byzantine agreement protocols. *Information and Computation*, 75(2):130–143, 1987.
- [DSW22] Brijesh Dongol, Gerhard Schellhorn, and Heike Wehrheim. Weak Progressive Forward Simulation Is Necessary and Sufficient for Strong Observational Refinement. In Bartek Klin, Slawomir Lasota, and Anca Muscholl, editors, *33rd International Conference on Concurrency Theory (CONCUR 2022)*, volume 243 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 31:1–31:23, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [EMMS26] Constantin Enea, Rupak Majumdar, Harshit Jitendra Motwani, and V. R. Sathiyarayanan. Verifying almost-sure termination for randomized distributed algorithms. *Proc. ACM Program. Lang.*, 10(POPL):1412–1441, 2026.
- [FKT22] Luciano Freitas, Petr Kuznetsov, and Andrei Tonkikh. Distributed randomness from approximate agreement, 2022.
- [SA21] Gilad Stern and Ittai Abraham. Living with asynchrony: the gather protocol. <https://decentralizedthoughts.github.io/2021-03-26-living-with-asynchrony-the-gather-protocol>, 2021.
- [SA24] Gilad Stern and Ittai Abraham. Gather with binding and verifiability. <https://decentralizedthoughts.github.io/2024-01-09-gather-with-binding-and-verifiability/>, 2024.